

```

import torch
import torch.nn as nn

class HebbianAutoencoder(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.W_enc =
nn.Parameter(torch.randn(input_dim,
hidden_dim) * 0.1)
        self.W_dec =
nn.Parameter(torch.randn(hidden_dim,
input_dim) * 0.1)

    def forward(self, x):
        h = torch.relu(x @ self.W_enc) #
encoding
        x_rec = h @ self.W_dec        # decoding
        return x_rec, h

    def hebbian_update(self, x, lr=0.01):
        x_rec, h = self.forward(x)
        error = x - x_rec

```

```
# Hebbian на encoder
delta_W_enc = lr * torch.outer(x.T, h)
self.W_enc.data += delta_W_enc
```

```
# Hebbian на decoder
delta_W_dec = lr * torch.outer(h.T, x)
self.W_dec.data += delta_W_dec
```

```
# Oja normalization
self.W_enc.data *= 0.99
self.W_dec.data *= 0.99
```

```
# Тест
```

```
model = HebbianAutoencoder(784, 256)
x = torch.randn(32, 784)
```

```
for epoch in range(100):
    x_rec, h = model(x)
    loss = F.mse_loss(x_rec, x)
    model.hebbian_update(x)
    print(f"Epoch {epoch}, Loss:
```

{loss.item():.4f}")